Practical 3 — Responsive UI Design

Shared layout CSS

```
* {
    box-sizing: border-box;
}

body {
    min-height: 100vh;
    margin: 0;
    display: flex;
    flex-direction: column;
    font-family: Arial, sans-serif;
}

.container {
    width: 100%;
    max-width: 1140px;
    margin: 0 auto;
    padding: 28px 18px 48px;
    flex: 1;
}

.header-container {
    max-width: 1140px;
    margin: 0 auto;
    padding: 14px 24px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 24px;
    flex-wrap: wrap;
}

.main-nav ul {
    display: flex;
    align-items: center;
    gap: 6px;
    flex-wrap: wrap;
    list-style: none;
}

.card-grid,
.event-list {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 22px;
}

.contact-layout,
.profile-top {
    display: grid;
    grid-template-columns: 1fr 1.35fr;
    gap: 26px;
}

.info-grid {
    display: grid;
    grid-template-columns: 130px 1fr;
    gap: 14px 16px;
}

.quick-access-content {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 24px;
```

```
    flex-wrap: wrap;
}

@media (max-width: 900px) {
    .contact-layout,
    .profile-top {
        grid-template-columns: 1fr;
    }

    .info-grid {
        grid-template-columns: 110px 1fr;
    }
}

@media (max-width: 720px) {
    .header-container {
        padding: 12px 18px;
    }

    .main-nav ul {
        gap: 4px;
    }

    .container {
        padding: 28px 18px 48px;
    }
}

@media (max-width: 520px) {
    .info-grid {
        grid-template-columns: 1fr;
        row-gap: 4px;
    }

    .footer-container {
        flex-direction: column;
        text-align: center;
    }

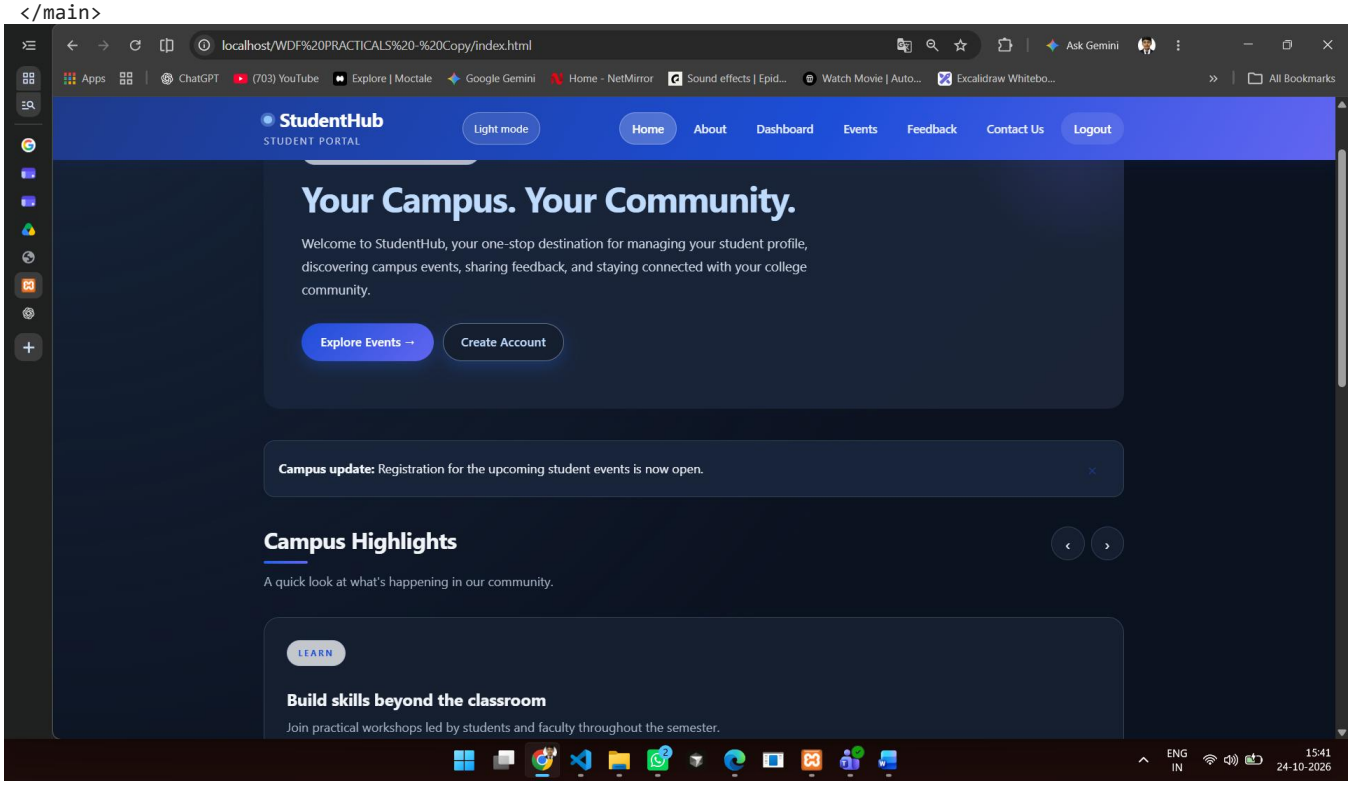
    .card-grid,
    .event-list {
        grid-template-columns: minmax(0, 1fr);
    }
}
```

Home page layout

```
<main class="container">
    <section class="hero">
        <div class="hero-content">
            <span class="hero-label">WELCOME TO YOUR CAMPUS</span>
            <h2>Your Campus. Your Community.</h2>
            <p>
                Manage your student profile, discover campus events,
                share feedback, and stay connected.
            </p>
            <div class="hero-actions">
                <a href="event.html" class="primary-button">Explore Events</a>
                <a href="register.html" class="secondary-button">Create Account</a>
            </div>
        </div>
    </section>

    <section class="features-section">
        <h2 class="page-title">Everything You Need</h2>
        <div class="card-grid">
            <article class="card">
                <h3>Your Profile</h3>
                <p>Keep your student information organized.</p>
                <a href="profile.html" class="card-link">View Dashboard</a>
            </article>
            <article class="card">
```

```
                <h3>Campus Events</h3>
                <p>Discover activities and opportunities on campus.</p>
                <a href="event.html" class="card-link">Discover Events</a>
            </article>
            <article class="card">
                <h3>Share Feedback</h3>
                <p>Help improve your student experience.</p>
                <a href="feedback.html" class="card-link">Give Feedback</a>
            </article>
        </div>
    </section>
</main>
```

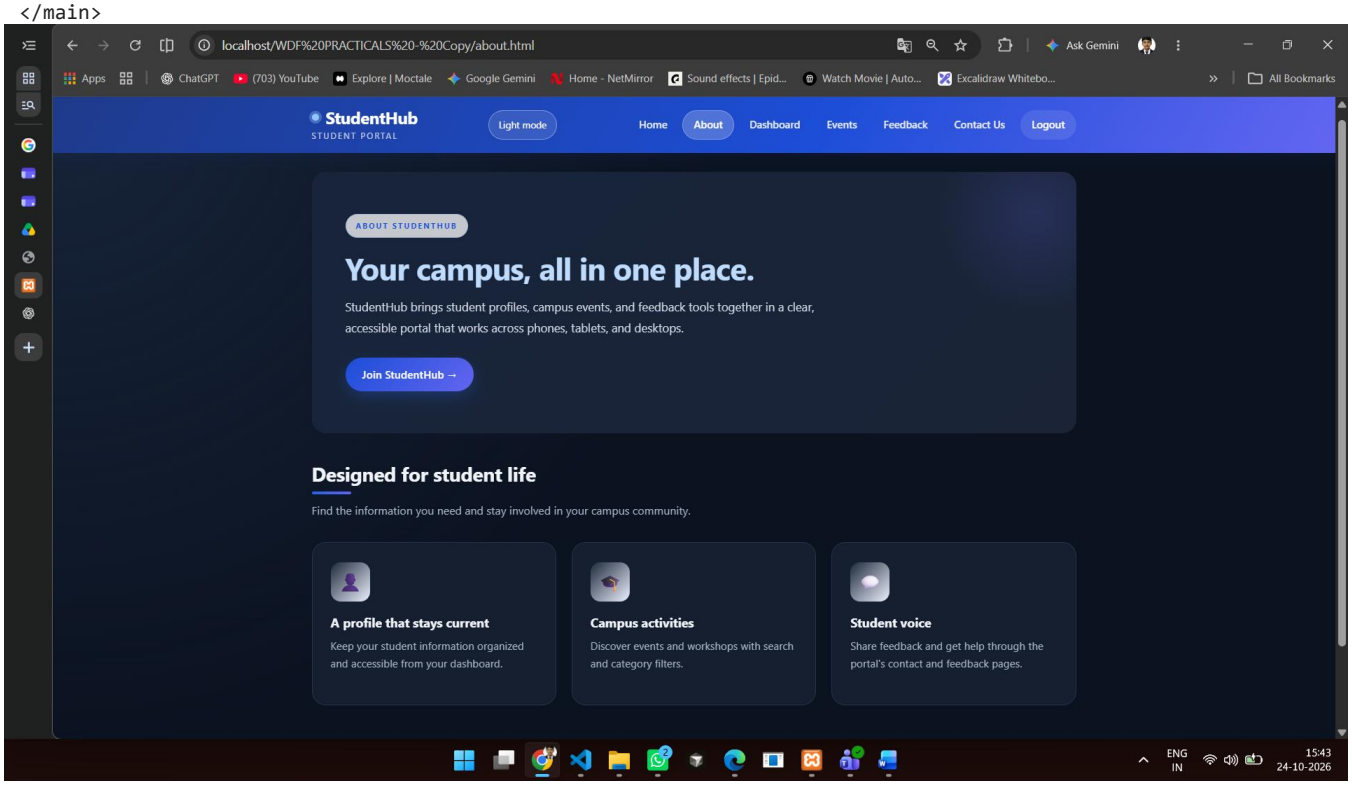


About page layout

```
<main class="container">
    <section class="hero">
        <div class="hero-content">
            <span class="hero-label">ABOUT STUDENTHUB</span>
            <h2>Your campus, all in one place.</h2>
            <p>
                StudentHub brings student profiles, campus events, and
                feedback tools together in one responsive portal.
            </p>
            <a href="register.html" class="primary-button">Join StudentHub</a>
        </div>
    </section>

    <section class="features-section">
        <h2 class="page-title">Designed for student life</h2>
        <div class="card-grid">
            <article class="card">
                <h3>A profile that stays current</h3>
                <p>Keep your student information organized.</p>
            </article>
            <article class="card">
                <h3>Campus activities</h3>
                <p>Discover events and workshops.</p>
            </article>
            <article class="card">
                <h3>Student voice</h3>
                <p>Share feedback and get help.</p>
            </article>
        </div>
```

```
    </section>
</main>
```

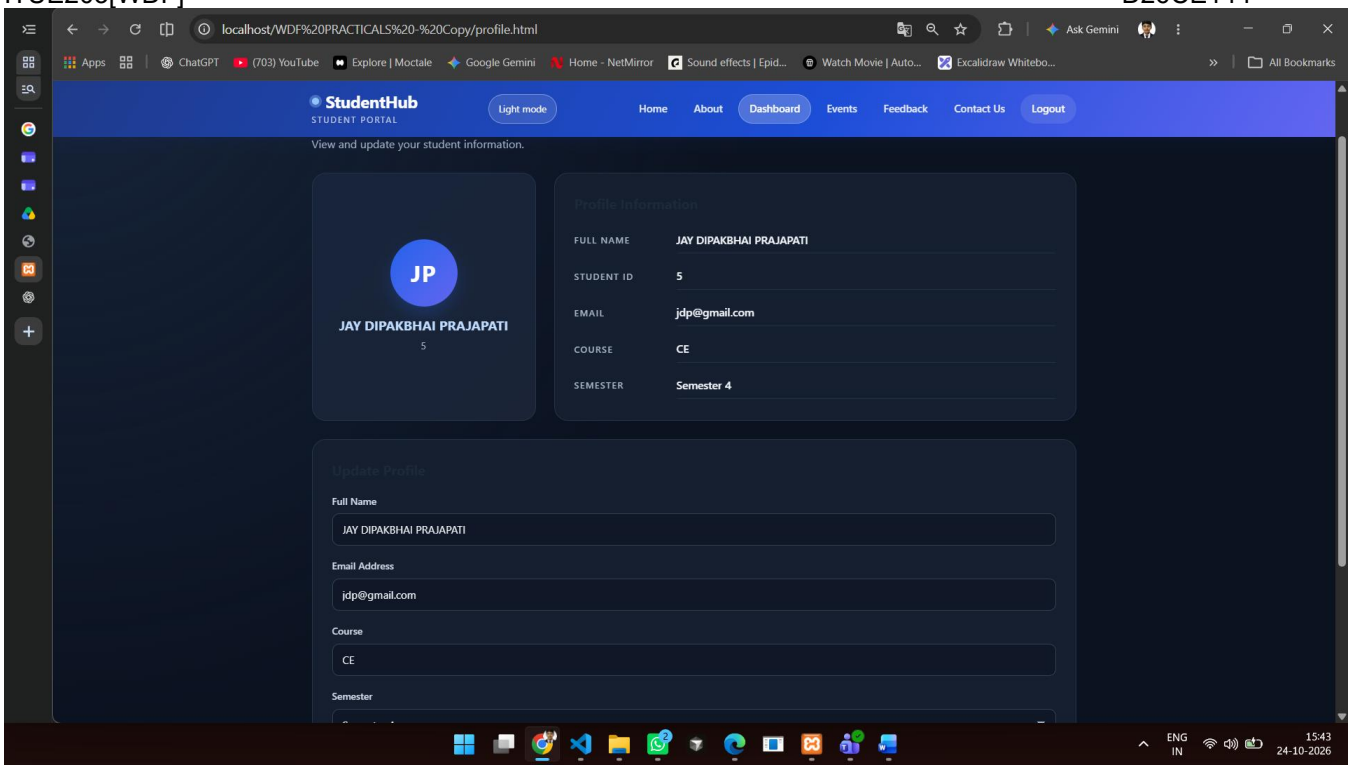


Dashboard layout

```
<main class="container">
    <div class="page-heading">
        <h2>Student Dashboard</h2>
        <p>View and update your student information.</p>
    </div>

    <section class="profile-top">
        <article class="profile-card">
            <div class="profile-placeholder" id="profileInitials">ST</div>
            <h3 id="displayName">Student Name</h3>
            <p id="displayStudentId">Student ID</p>
        </article>

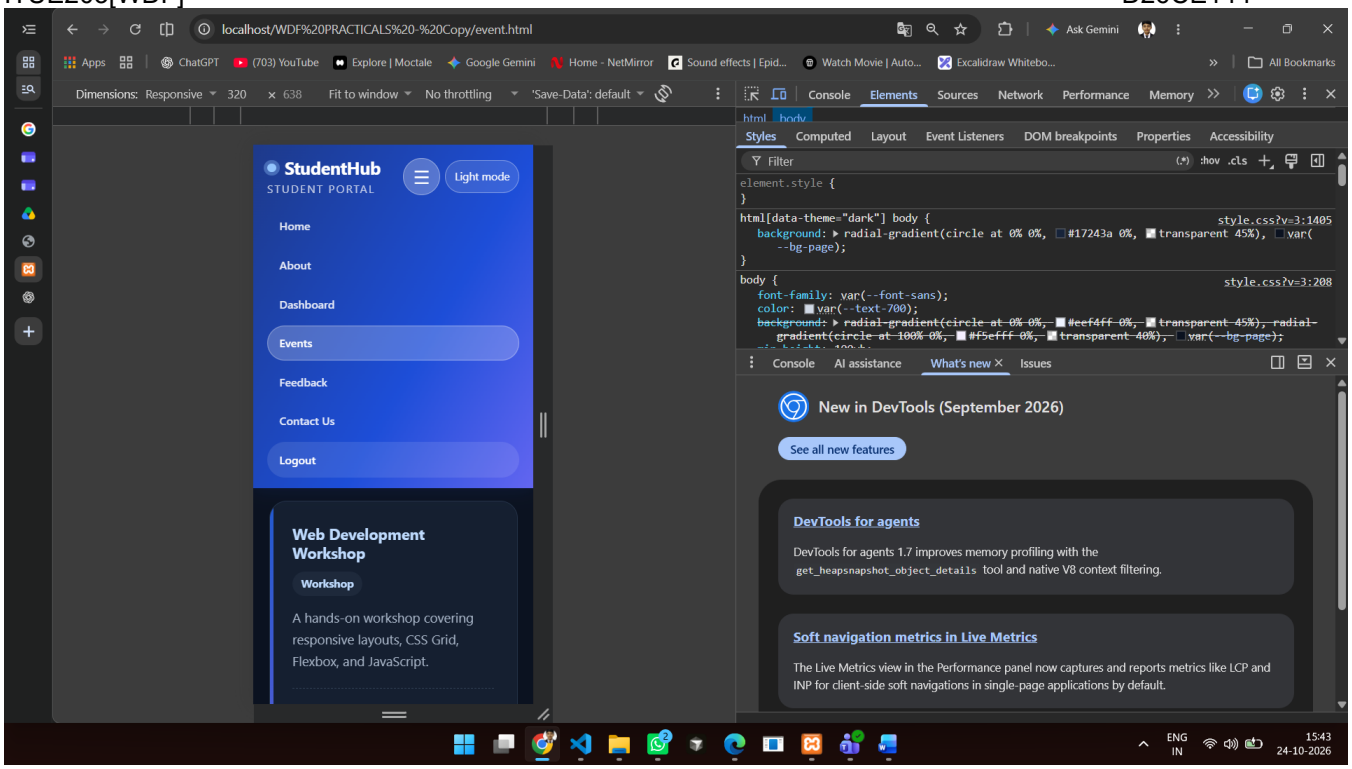
        <article class="information-card">
            <h3>Profile Information</h3>
            <div class="info-grid">
                <div class="info-label">Full Name</div>
                <div class="info-value" id="infoName">Student Name</div>
                <div class="info-label">Email</div>
                <div class="info-value" id="infoEmail">student@example.com</div>
                <div class="info-label">Course</div>
                <div class="info-value" id="infoCourse">Course</div>
            </div>
        </article>
    </section>
</main>
```



Mobile hamburger navigation — HTML

```
<div class="header-actions">
    <button class="menu-toggle"
            type="button"
            aria-controls="site-navigation"
            aria-expanded="false">
        <span aria-hidden="true">☰</span>
        <span class="visually-hidden">Toggle navigation</span>
    </button>
    <button class="theme-toggle" type="button" aria-pressed="false">
        Dark mode
    </button>
</div>

<nav class="main-nav" id="site-navigation" aria-label="Main navigation">
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="profile.html">Dashboard</a></li>
        <li><a href="event.html">Events</a></li>
    </ul>
</nav>
```



Mobile hamburger navigation — CSS

```
.menu-toggle {
    display: none;
}

@media (max-width: 720px) {
    .menu-toggle {
        display: inline-flex;
    }

    .main-nav {
        display: none;
        flex: 0 0 100%;
        width: 100%;
    }

    .main-nav.is-open {
        display: block;
    }

    .main-nav ul {
        align-items: stretch;
        flex-direction: column;
    }

    .main-nav a {
        display: block;
        padding: 10px 14px;
    }
}
```

Practical 4 — JavaScript DOM Manipulation and UI Interactivity

Homepage interactive components — HTML

```
<aside class="notice-banner" id="noticeBanner" role="status">
    <p><strong>Campus update:</strong> Event registration is open.</p>
    <button class="notice-dismiss" type="button" aria-label="Dismiss notification">
        &times;
    </button>
</aside>

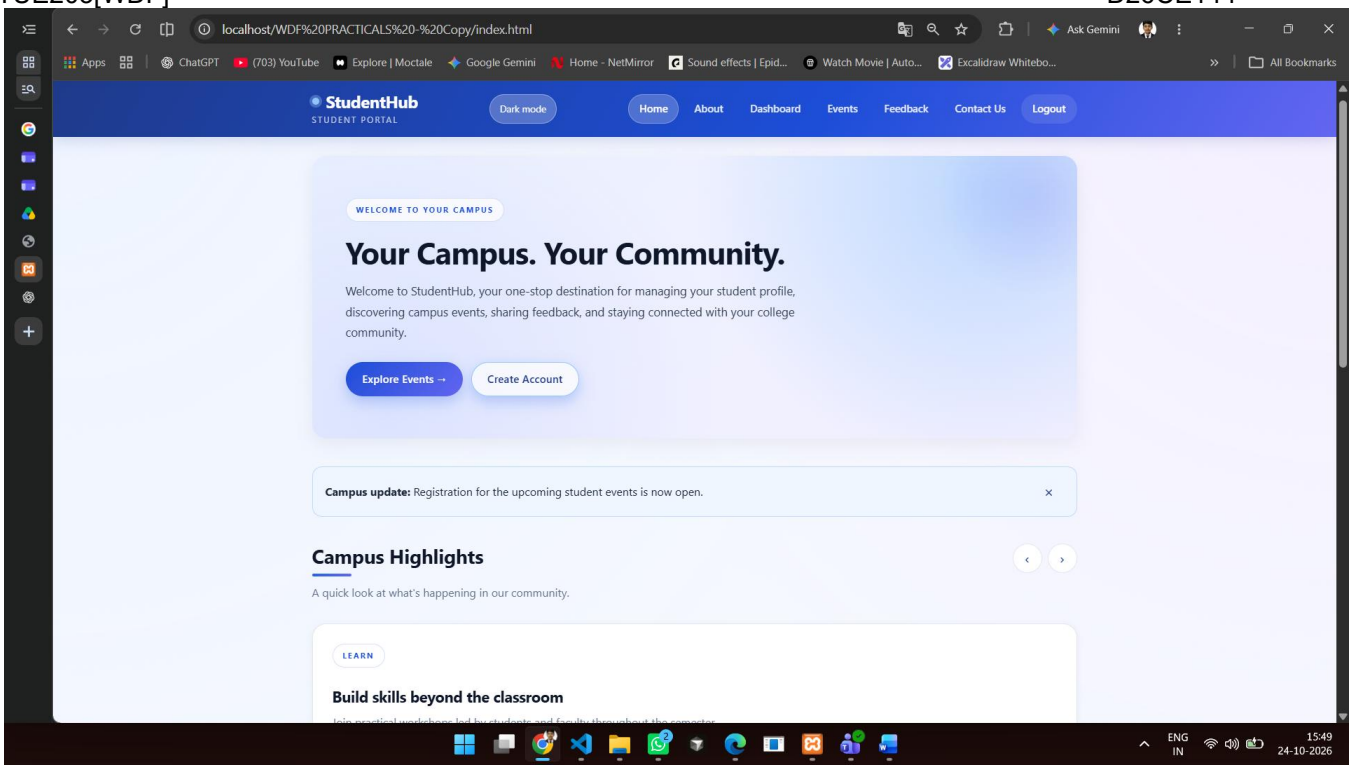
<section class="highlights-section" aria-labelledby="highlights-title">
    <h2 id="highlights-title">Campus Highlights</h2>

    <div class="slider-controls">
        <button type="button" data-slide="previous" aria-label="Previous highlight">‹</button>
        <button type="button" data-slide="next" aria-label="Next highlight">›</button>
    </div>

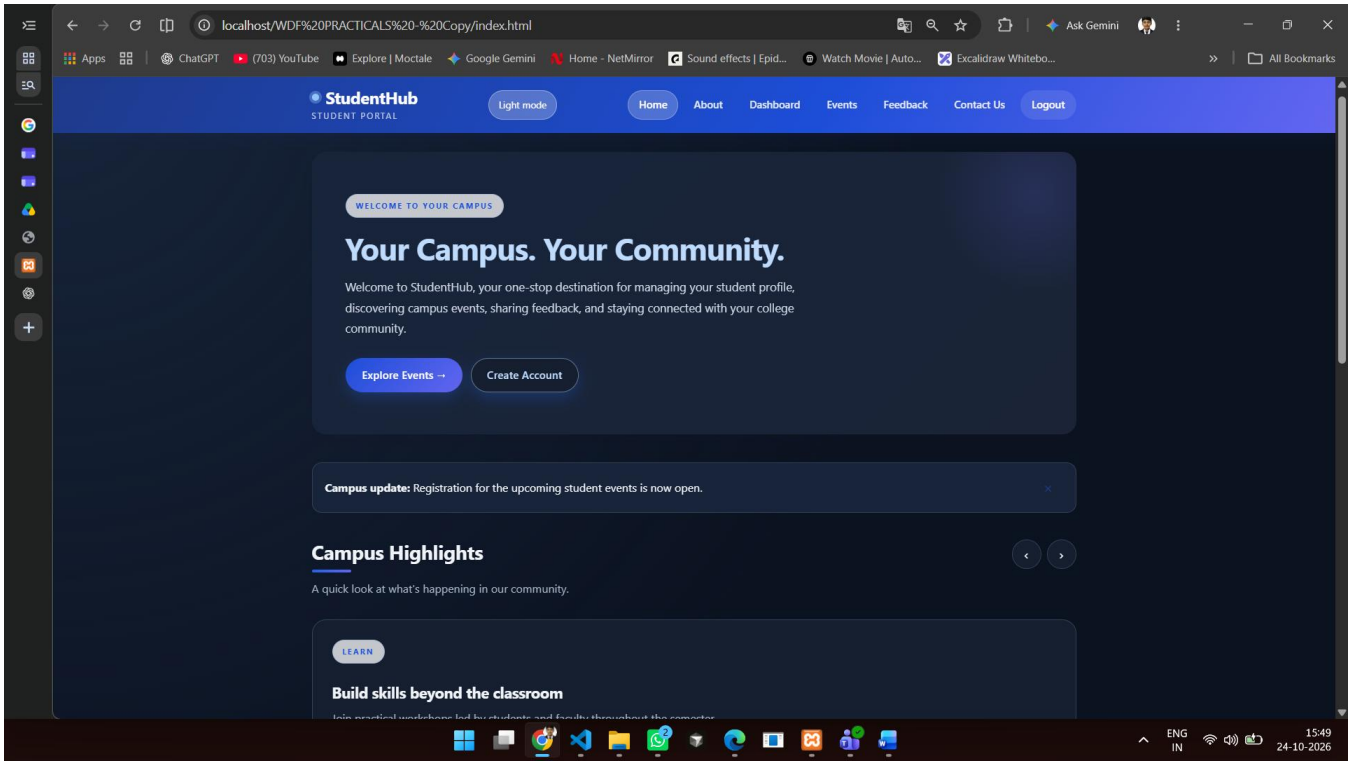
    <div class="highlight-slider" aria-live="polite">
        <article class="highlight-slide is-active">
            <h3>Build skills beyond the classroom</h3>
            <p>Join practical campus workshops.</p>
        </article>
        <article class="highlight-slide">
            <h3>Find your campus community</h3>
            <p>Meet classmates and discover student activities.</p>
        </article>
        <article class="highlight-slide">
            <h3>Your ideas help shape the experience</h3>
            <p>Share feedback with StudentHub.</p>
        </article>
    </div>
</section>

<section class="faq-section" aria-labelledby="faq-title">
    <h2 id="faq-title">Frequently Asked Questions</h2>
    <details class="faq-item">
        <summary>How do I update my student information?</summary>
        <p>Open your Dashboard and use the Update Profile form.</p>
    </details>
    <details class="faq-item">
        <summary>Where can I find upcoming activities?</summary>
        <p>Visit the Events page to search and filter campus activities.</p>
    </details>
</section>

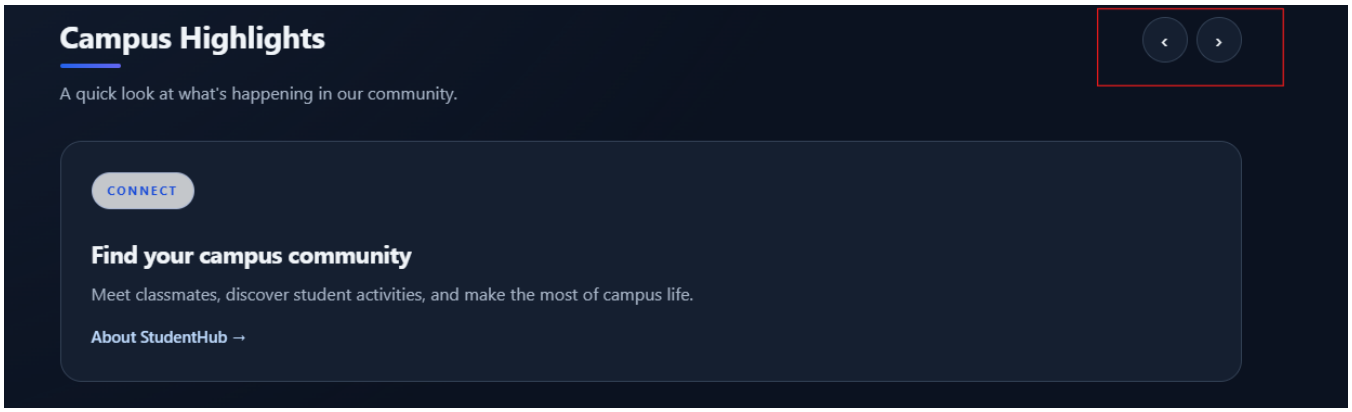
<dialog class="welcome-modal" id="welcomeModal" aria-labelledby="modal-title">
    <button class="modal-close" type="button" aria-label="Close welcome message">
        &times;
    </button>
    <h2 id="modal-title">Welcome to your campus community</h2>
    <p>Discover events and stay connected with StudentHub.</p>
    <a class="primary-button" href="event.html">Explore events</a>
</dialog>
```

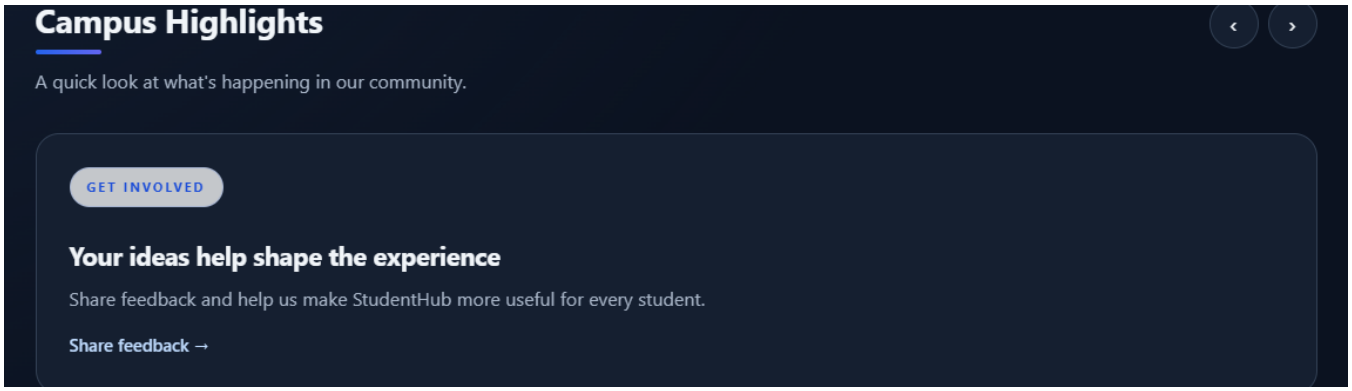


Light mode



Dark mode





Slider

JavaScript interactions

```
document.addEventListener("DOMContentLoaded", () => {
    const themeButtons = document.querySelectorAll(".theme-toggle");

    function setTheme(theme) {
        document.documentElement.dataset.theme = theme;
        localStorage.setItem("studenthub-theme", theme);

        themeButtons.forEach((button) => {
            const isDark = theme === "dark";
            button.textContent = isDark ? "Light mode" : "Dark mode";
            button.setAttribute("aria-pressed", String(isDark));
        });
    }

    setTheme(localStorage.getItem("studenthub-theme") === "dark" ? "dark" : "light");

    themeButtons.forEach((button) => {
        button.addEventListener("click", () => {
            const nextTheme =
                document.documentElement.dataset.theme === "dark" ? "light" : "dark";
            setTheme(nextTheme);
        });
    });

    document.querySelectorAll(".menu-toggle").forEach((button) => {
        const navigation = document.getElementById(button.getAttribute("aria-controls"));

        if (!navigation) return;

        button.addEventListener("click", () => {
            const isOpen = navigation.classList.toggle("is-open");
            button.setAttribute("aria-expanded", String(isOpen));
```

```
        });
    });

    const notice = document.getElementById("noticeBanner");
    notice?.querySelector(".notice-dismiss")?.addEventListener("click", () => {
        notice.hidden = true;
    });

    const slides = [...document.querySelectorAll(".highlight-slide")];
    let activeSlide = slides.findIndex((slide) => slide.classList.contains("is-active"));

    document.querySelectorAll("[data-slide]").forEach((button) => {
        button.addEventListener("click", () => {
            if (slides.length === 0) return;

            slides[activeSlide].classList.remove("is-active");
            const step = button.dataset.slide === "next" ? 1 : -1;
            activeSlide = (activeSlide + step + slides.length) % slides.length;
            slides[activeSlide].classList.add("is-active");
        });
    });

    const modal = document.getElementById("welcomeModal");
    const closeButton = modal?.querySelector(".modal-close");

    if (modal instanceof HTMLDialogElement &&
        !sessionStorage.getItem("welcome-seen")) {
        window.setTimeout(() => {
            if (!modal.open) modal.showModal();
        }, 900);

        modal.addEventListener("close", () => {
            sessionStorage.setItem("welcome-seen", "true");
        });

        closeButton?.addEventListener("click", () => modal.close());
    }
});
```

Practical 5 — Registration Form and Validation

Registration form — HTML

```
<form id="registerForm" action="php/register.php" method="POST" novalidate>
    <div class="form-group">
        <label for="fullname">Full Name</label>
        <input type="text" id="fullname" name="fullname"
               autocomplete="name" required>
    </div>

    <div class="form-group">
        <label for="register-email">Email Address</label>
        <input type="email" id="register-email" name="email"
               autocomplete="email" required>
    </div>

    <div class="form-group">
        <label for="mobile">Mobile Number</label>
        <input type="tel" id="mobile" name="mobile"
               inputmode="numeric" maxlength="10" required>
```

```
    </div>

    <div class="form-group">
        <label for="course">Course</label>
        <input type="text" id="course" name="course" required>
    </div>

    <div class="form-group">
        <label for="year">Year of Study</label>
        <select id="year" name="year" required>
            <option value="">Select Year</option>
            <option value="1">First Year</option>
            <option value="2">Second Year</option>
            <option value="3">Third Year</option>
            <option value="4">Fourth Year</option>
        </select>
    </div>

    <div class="form-group">
        <label for="register-password">Password</label>
        <input type="password" id="register-password" name="password"
               minlength="8" autocomplete="new-password" required>
    </div>

    <div class="form-group">
        <label for="confirm-password">Confirm Password</label>
        <input type="password" id="confirm-password" name="confirm-password"
               autocomplete="new-password" required>
    </div>

    <fieldset class="choice-group">
        <legend>Gender</legend>
        <label><input type="radio" name="gender" value="female" required> Female</label>
        <label><input type="radio" name="gender" value="male"> Male</label>
        <label><input type="radio" name="gender" value="other"> Other</label>
        <label><input type="radio" name="gender" value="prefer-not-to-say">
            Prefer not to say
        </label>
    </fieldset>

    <div class="terms-group">
        <label for="terms">
            <input type="checkbox" id="terms" name="terms"
                   value="accepted" required>
            I accept the terms and conditions.
        </label>
    </div>

    <button type="submit" class="submit-button">Create Account</button>
    <p id="registerMessage" class="form-message" role="alert"></p>
</form>
```

JavaScript validation

```
document.addEventListener("DOMContentLoaded", () => {
    const form = document.getElementById("registerForm");
    const message = document.getElementById("registerMessage");

    if (!form || !message) return;

    const rules = [
        {
            name: "fullname",
            pattern: /^[\p{L}][\p{L}\s'.-]*$/u,
            message: "Enter a valid name."
        },
        {
            name: "email",
            pattern: /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/,
            message: "Enter a valid email address."
        },
```

```
        {
            name: "mobile",
            pattern: /^[6-9]\d{9}$/,
            message: "Enter a valid 10-digit mobile number."
        },
        {
            name: "course",
            pattern: /^[\p{L}\p{N}][\p{L}\p{N}\s&'().-]*$/u,
            message: "Enter a valid course name."
        },
        {
            name: "year",
            pattern: /^[1-4]$/,
            message: "Select your year of study."
        },
        {
            name: "password",
            pattern: /^(?=.*[A-Za-z])(?=.*\d).{8,}$/,
            message: "Password must be at least 8 characters and include a letter and a number."
        }
    ];

    function showError(text, field) {
        message.textContent = text;
        message.className = "form-message error";
        field?.focus();
    }

    form.addEventListener("submit", async (event) => {
        event.preventDefault();
        message.textContent = "";

        for (const rule of rules) {
            const field = form.elements.namedItem(rule.name);

            if (!rule.pattern.test(field.value.trim())) {
                showError(rule.message, field);
                return;
            }
        }

        const password = form.elements.namedItem("password");
        const confirmPassword = form.elements.namedItem("confirm-password");
        const gender = form.querySelector('input[name="gender"]:checked');
        const terms = document.getElementById("terms");

        if (password.value !== confirmPassword.value) {
            showError("Passwords do not match.", confirmPassword);
            return;
        }

        if (!gender) {
            showError("Select a gender option to continue.",
                      form.querySelector('input[name="gender"]'));
            return;
        }

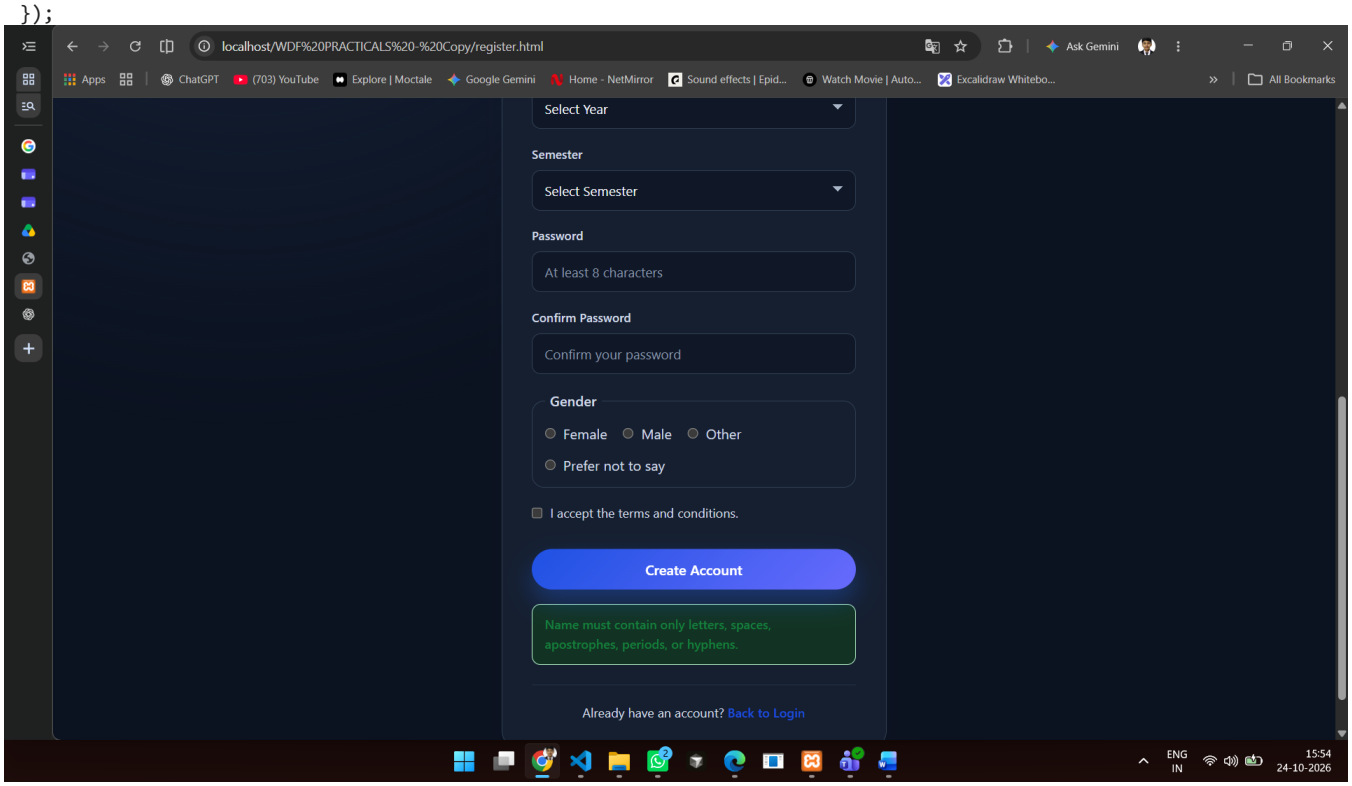
        if (!terms.checked) {
            showError("Please accept the terms and conditions.", terms);
            return;
        }

        try {
            const response = await fetch(form.action, {
                method: "POST",
                body: new FormData(form)
            });
            const result = await response.json();

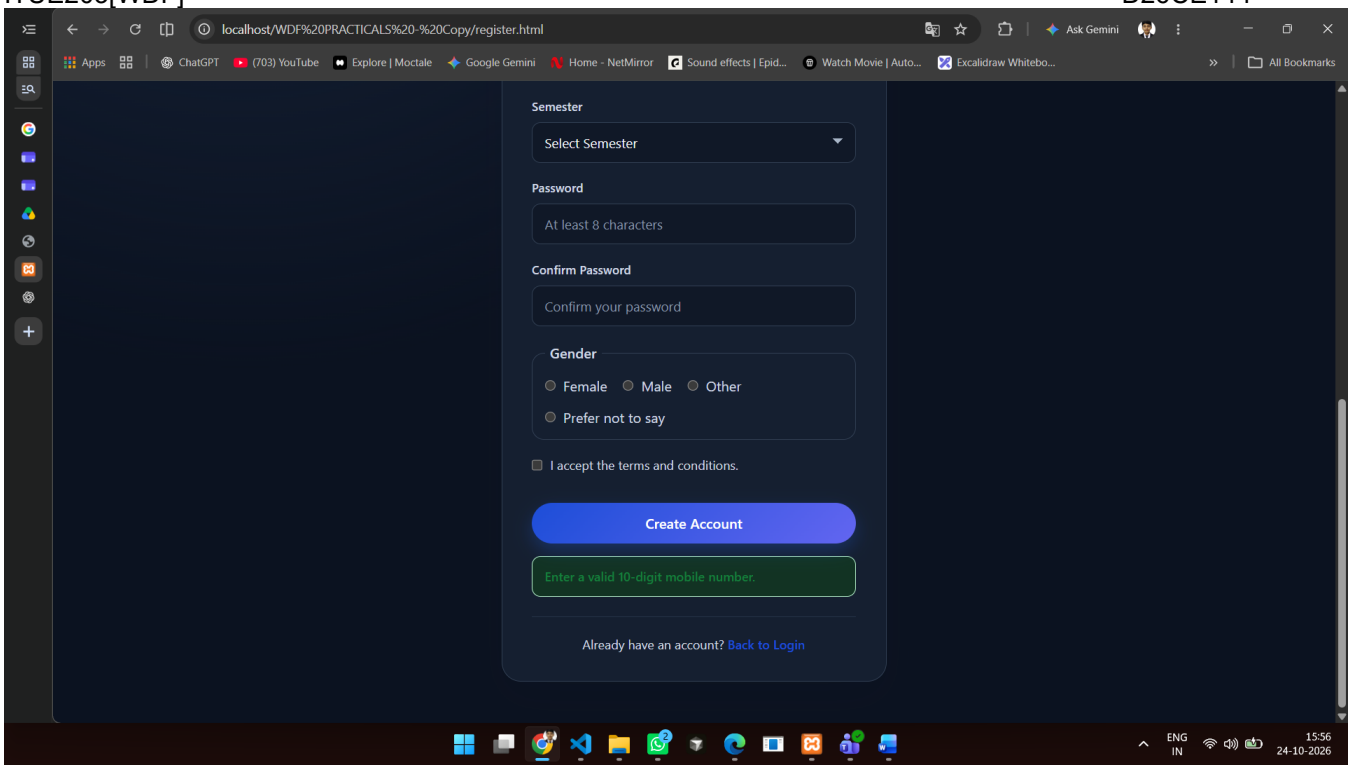
            if (!result.success) {
                showError(result.message, null);
                return;
```

```
            }

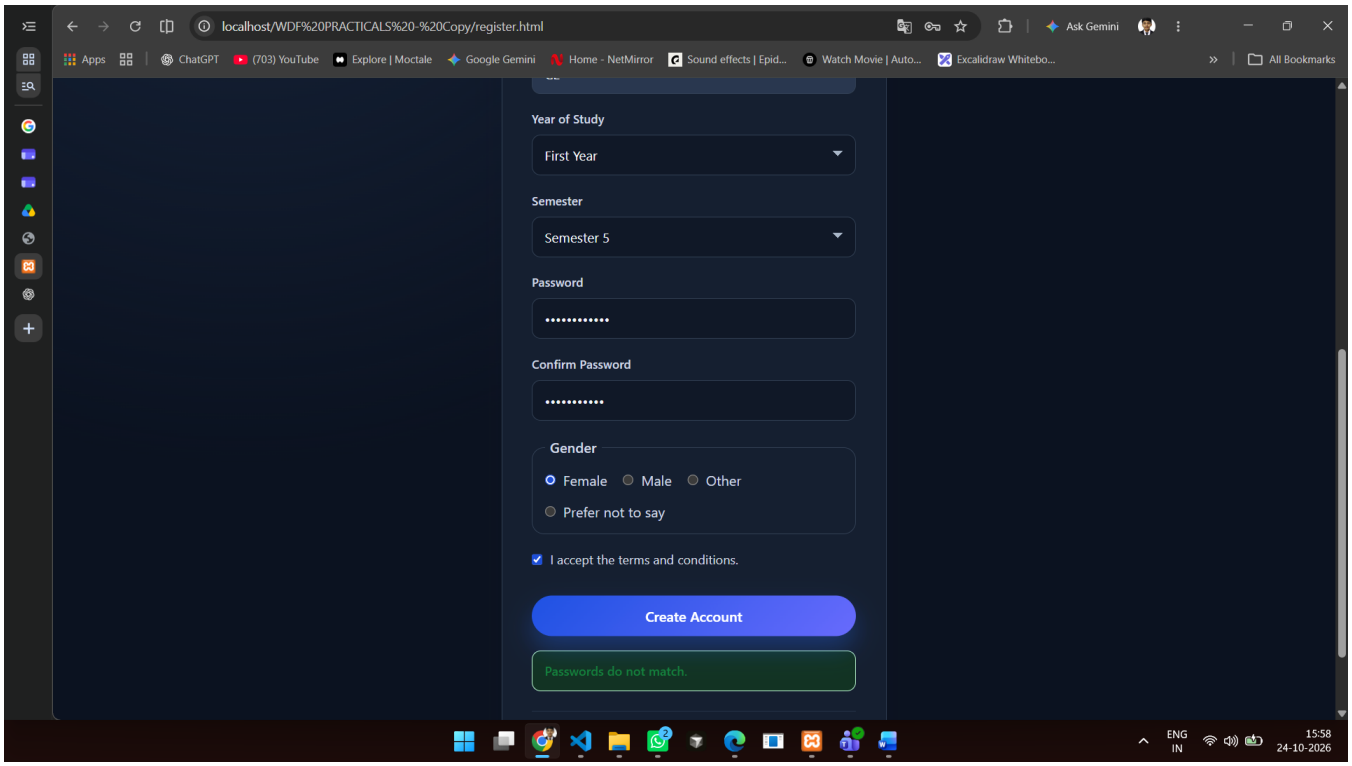
            message.textContent = "Account created successfully.";
            message.className = "form-message success";
            form.reset();
        } catch (error) {
            console.error(error);
            showError("Unable to connect to the server. Please try again.", null);
        }
    });
});
```



NAME ERROR



MOBILE ERROR



PASSWORD ERROR

Practical 6 — Fetch JSON, Search, Filter, Sort, and Pagination

Events page controls — HTML

```
<section class="event-tools" aria-label="Search and filter events">
    <div class="form-group">
        <label for="eventSearch">Search events</label>
        <input type="search" id="eventSearch"
               placeholder="Search by name, venue, or description">
    </div>

    <div class="form-group">
        <label for="eventCategory">Category</label>
        <select id="eventCategory">
            <option value="all">All categories</option>
        </select>
    </div>

    <div class="form-group">
        <label for="eventSort">Sort by</label>
        <select id="eventSort">
            <option value="date-asc">Date: soonest first</option>
            <option value="date-desc">Date: latest first</option>
            <option value="title-asc">Name: A to Z</option>
            <option value="title-desc">Name: Z to A</option>
        </select>
    </div>
</section>

<p id="eventStatus" class="event-status" role="status">Loading events...</p>
<section class="event-list" id="eventList" aria-label="Upcoming student events"></section>
<nav class="pagination" id="eventPagination" aria-label="Event pages"></nav>

<script src="js/events.js"></script>
```

data/events.json

```
[
  {
    "title": "Student Orientation Program",
    "description": "Welcome program for students with information about academic and campus activities.",
    "date": "2026-11-05",
    "time": "10:00 AM",
    "venue": "Main Auditorium",
    "category": "Campus"
  },
  {
    "title": "Web Development Workshop",
    "description": "A hands-on workshop covering responsive layouts, CSS Grid, Flexbox, and JavaScript.",
    "date": "2026-11-12",
    "time": "11:00 AM",
    "venue": "Computer Laboratory",
    "category": "Workshop"
  },
  {
    "title": "Student Technology Seminar",
    "description": "Explore emerging technologies, student projects, and career opportunities.",
    "date": "2026-11-19",
    "time": "2:00 PM",
    "venue": "Seminar Hall",
    "category": "Seminar"
  },
  {
    "title": "Campus Cultural Evening",
    "description": "An evening of student performances, music, and cultural activities.",
    "date": "2026-11-26",
    "time": "5:30 PM",
    "venue": "Open-Air Theatre",
    "category": "Campus"
  },
  {
```

```
    "title": "Introduction to UI Design",
    "description": "Learn the principles of accessible, mobile-first interface design.",
    "date": "2026-12-03",
    "time": "1:00 PM",
    "venue": "Design Studio",
    "category": "Workshop"
  },
  {
    "title": "Student Project Showcase",
    "description": "Meet student teams and see projects created across the academic year.",
    "date": "2026-12-10",
    "time": "10:30 AM",
    "venue": "Innovation Centre",
    "category": "Showcase"
  },
  {
    "title": "Career Readiness Seminar",
    "description": "Practical guidance on resumes, interviews, and preparing for internships.",
    "date": "2026-12-17",
    "time": "2:30 PM",
    "venue": "Seminar Hall",
    "category": "Seminar"
  }
]
```

js/events.js

```
document.addEventListener("DOMContentLoaded", async () => {
    const eventList = document.getElementById("eventList");
    const eventStatus = document.getElementById("eventStatus");
    const searchInput = document.getElementById("eventSearch");
    const categorySelect = document.getElementById("eventCategory");
    const sortSelect = document.getElementById("eventSort");
    const pagination = document.getElementById("eventPagination");

    if (!eventList || !eventStatus || !searchInput ||
        !categorySelect || !sortSelect || !pagination) {
        return;
    }

    const pageSize = 3;
    let events = [];
    let currentPage = 1;

    function createMeta(label, value) {
        const paragraph = document.createElement("p");
        const strong = document.createElement("strong");

        strong.textContent = `${label}: `;
        paragraph.append(strong, document.createTextNode(value));

        return paragraph;
    }

    function getVisibleEvents() {
        const searchTerm = searchInput.value.trim().toLocaleLowerCase();
        const selectedCategory = categorySelect.value;

        const filteredEvents = events
            .filter((item) =>
                selectedCategory === "all" ||
                item.category === selectedCategory
            )
            .filter((item) =>
                [item.title, item.description, item.venue, item.category]
                    .some((value) =>
                        value.toLocaleLowerCase().includes(searchTerm)
                    )
            );

        filteredEvents.sort((first, second) => {
```

```
            switch (sortSelect.value) {
                case "date-desc":
                    return second.date.localeCompare(first.date);
                case "title-asc":
                    return first.title.localeCompare(second.title);
                case "title-desc":
                    return second.title.localeCompare(first.title);
                default:
                    return first.date.localeCompare(second.date);
            }
        });

        return filteredEvents;
    }

    function render() {
        const visibleEvents = getVisibleEvents();
        const pageCount = Math.ceil(visibleEvents.length / pageSize);

        currentPage = Math.min(currentPage, Math.max(pageCount, 1));

        const pageEvents = visibleEvents.slice(
            (currentPage - 1) * pageSize,
            currentPage * pageSize
        );

        eventList.replaceChildren();
        pagination.replaceChildren();

        if (visibleEvents.length === 0) {
            eventStatus.textContent =
                "No events match your search. Try another keyword or category.";
            return;
        }

        eventStatus.textContent =
            `Showing ${pageEvents.length} of ${visibleEvents.length} events.`;

        pageEvents.forEach((item) => {
            const article = document.createElement("article");
            article.className = "event-card";

            const title = document.createElement("h3");
            title.textContent = item.title;

            const description = document.createElement("p");
            description.textContent = item.description;

            const category = document.createElement("span");
            category.className = "event-category";
            category.textContent = item.category;

            const meta = document.createElement("div");
            meta.className = "event-meta";
            meta.append(
                createMeta("Date", new Intl.DateTimeFormat(undefined, {
                    day: "numeric",
                    month: "long",
                    year: "numeric"
                }).format(new Date(`${item.date}T00:00:00`))),
                createMeta("Time", item.time),
                createMeta("Venue", item.venue)
            );

            article.append(title, category, description, meta);
            eventList.append(article);
        });

        for (let page = 1; page <= pageCount; page += 1) {
            const button = document.createElement("button");
            button.type = "button";
```

```
            button.className = "pagination-button";
            button.textContent = String(page);
            button.setAttribute("aria-label", `Page ${page}`);
            button.setAttribute("aria-current", String(page === currentPage));
            button.disabled = page === currentPage;

            button.addEventListener("click", () => {
                currentPage = page;
                render();
            });

            pagination.append(button);
        }
    }

    try {
        const response = await fetch("data/events.json");

        if (!response.ok) {
            throw new Error(`Event request failed: ${response.status}`);
        }

        const data = await response.json();

        if (!Array.isArray(data)) {
            throw new Error("Event data must be a JSON array.");
        }

        events = data.filter((item) =>
            item &&
            typeof item.title === "string" &&
            typeof item.description === "string" &&
            typeof item.date === "string" &&
            typeof item.time === "string" &&
            typeof item.venue === "string" &&
            typeof item.category === "string"
        );

        [...new Set(events.map((item) => item.category))]
            .sort()
            .forEach((category) => {
                const option = document.createElement("option");
                option.value = category;
                option.textContent = category;
                categorySelect.append(option);
            });

        if (events.length === 0) {
            throw new Error("No valid event records were found.");
        }

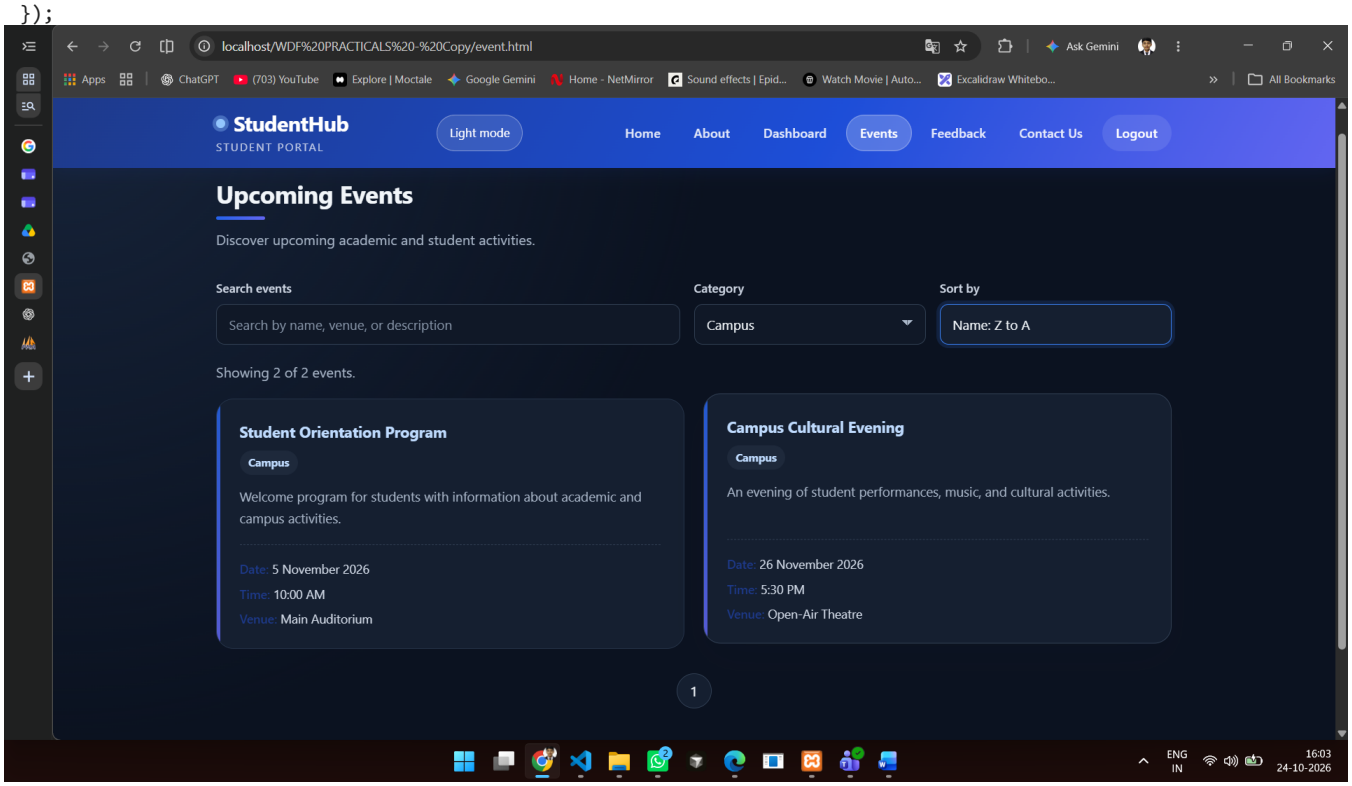
        searchInput.addEventListener("input", () => {
            currentPage = 1;
            render();
        });

        categorySelect.addEventListener("change", () => {
            currentPage = 1;
            render();
        });

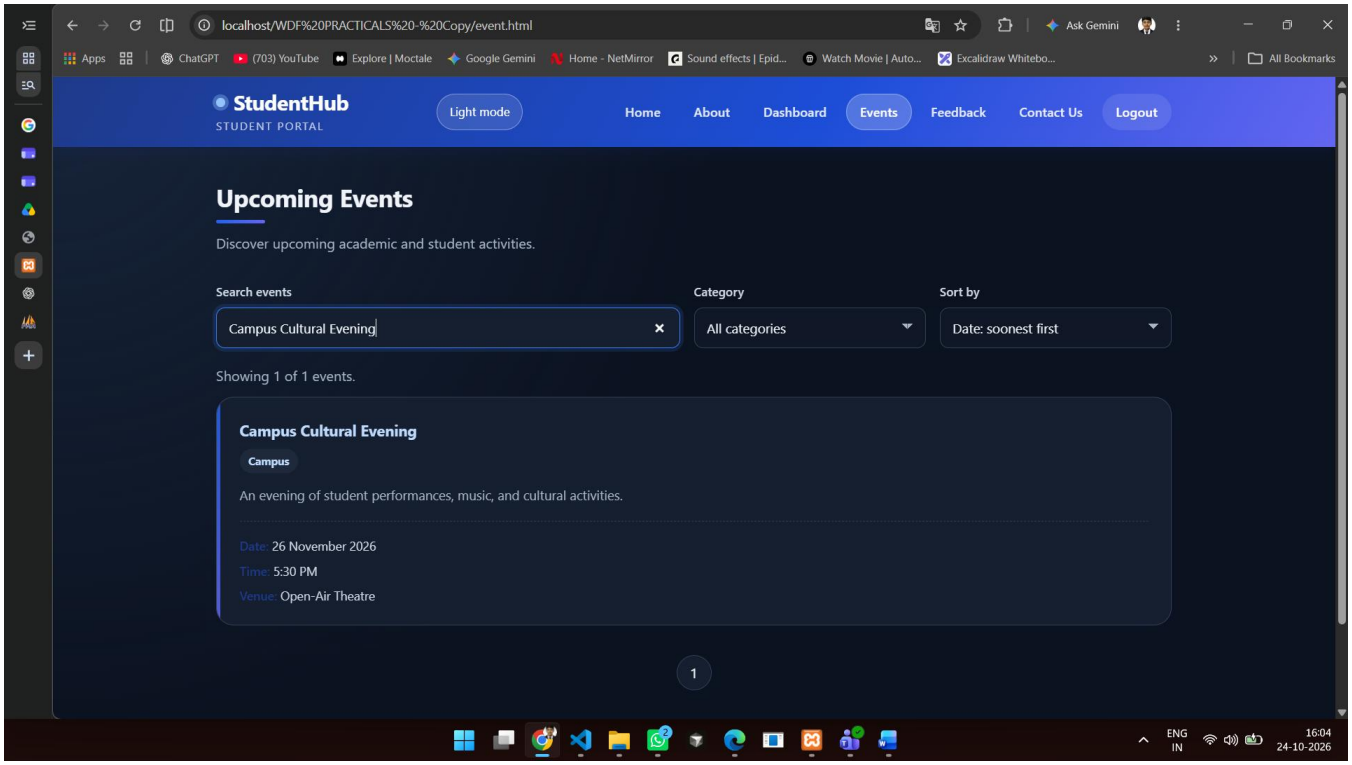
        sortSelect.addEventListener("change", () => {
            currentPage = 1;
            render();
        });

        render();
    } catch (error) {
        console.error(error);
        eventStatus.textContent =
            "Events could not be loaded. Please try again later.";
```

```
            eventStatus.classList.add("error");
        }
    });
```



FILTER CATGORY AND SORT WISE



NAME WISE